Characterizing Floating-Point Validation Rules for Tensor Superoptimization

Ayaan Faisal
Rutgers University

August 19, 2026

Abstract

Tensor superoptimizers verify candidate equivalence over exact or idealized arithmetic, then accept the compiled floating-point kernel by comparing it against a reference implementation on random inputs, elementwise, within $|g_i - b_i| \le \texttt{atol} + \texttt{rtol}\,|b_i|$. Only Axon states its constants ($\texttt{rtol} = \texttt{atol} = 10^{-4}$ on FP32), and no system fully specifies the reference, draw protocol, and precision scope together. We measure what the unstated coordinates cost, on six rewrites that are exact identities over the reals and eighteen injected bugs. The rule's verdict moves with every coordinate we vary. In float32 PyTorch on an Apple M3 (CPU), with both operands drawn i.i.d. as a standalone kernel validator draws them, a valid tiled matmul is rejected on 48/100 draws at K=4096 and 100/100 at K=11008, the contraction widths of Llama-2-7B, and the onset moves fourfold with input scale. An exact envelope analysis shows that no $(\texttt{atol}, \texttt{rtol})$ pair separates the recorded corpus under either extreme cross-draw semantics, all-draws-must-pass or any-draw-may-pass, at any $\texttt{rtol} \ge 0$; a real eps-placement bug evades every recorded draw at the published constant. At fp16 on one registered cell, the choice among three references flips the verdict and the direction of the accuracy comparison. Real-activation FP32 cells pass everywhere measured. These results do not demonstrate failures in the measured systems. They identify the coordinates that must be stated for numerical validation to be reproducible.

1 Introduction

A kernel autotuner tunes a fixed computation against a template that pins semantics; one reference validates every candidate. A superoptimizer searches program structure across millions of candidates, and the search selects for whatever passes the acceptance rule. Mirage [1], Prism [2], and Axon [3] verify candidate equivalence over exact or idealized arithmetic (Section 2) and close the distance to the compiled floating-point kernel with a sampled comparison against a reference implementation. Axon and Mirage state this limitation; Prism reports random equivalence testing without a stated criterion and does not otherwise discuss floating-point behavior. None fully specifies the reference, threshold, draw protocol, and precision scope together, so a reader cannot compute what the check accepts or rejects.

A tolerance pair does not determine a floating-point validation verdict. The protocol does, and it has several independent coordinates: the reference implementation, the precision, the input distribution and scale, the per-element comparison rule, the within-tensor aggregation (one violating element rejects the tensor), the number of independent draws, and the cross-draw aggregation rule. We vary the reference, precision, input distribution, scale, tolerance constants, draw count, and cross-draw semantics; the verdict moves with each. We hold the per-element functional form and the within-tensor aggregation fixed and isolate their roles analytically (F1).

We measure two failure directions on the same rule at the same tolerance: the rule rejects rewrites that are algebraic identities over the reals, and it accepts injected bugs. We find no evidence that any system has shipped a wrong kernel.

Scope: we characterize the rule, not the systems' outputs. The rewrites are PyTorch reimplementations of transformations these systems accept, with per-pair provenance recorded in the artifact. (The online-softmax rescale form is unclassified by all three source papers and is in the corpus for that reason, F7.) They run on an Ampere GPU and an Apple M3 with TF32 disabled. No kernel emitted by Mirage, Prism, or Axon is executed, and Axon's deployment target (Trainium/NKI) differs from our backend in accumulation and lowering. A finding transfers to another backend only insofar as the rule drives it; Section 5 marks which findings that caveat touches.

"Valid" below means equivalent over the reals; "bug" means inequivalent over the reals. Real-equivalence does not imply numerical acceptability. A real identity can be numerically unstable, and a semantic bug can be numerically negligible on a given input domain. Our corpus contains both cases, and Section 4 keeps the two axes separate.

Contributions:

1. A two-class corpus: six real-equivalent rewrites with

recorded per-pair provenance (identities established symbolically; implementations validated to 3.5×10^{-15} against a 50-digit-validated float64 oracle), and eighteen injected bugs across six classes whose float64 divergences span 4.1×10^{-6} to 8.2×10^{-1}.

2. An instrument reporting, per cell, the floor, total, and differential errors against a float64 oracle, the direction ratio, and the rule's verdict under three references.

3. An exact corpus-level separability analysis under the extreme Boolean cross-draw semantics, characterized over the whole domain $\texttt{rtol} \geq 0$ (Appendix B), in addition to a 64-point tolerance grid.

4. An audit trail: pre-registration with dated amendments, predictions committed before every run, and a complete errata of our own corrected results.

2 Background

Mirage restricts search to its Lax fragment and probabilistically verifies candidate equivalence by evaluation over random finite-field values. The theorem bounds the probability of accepting a non-equivalent program. The implementation runs a single random test, which the authors acknowledge [1]. Its primes are stated in the paper as p=227 and q=113, chosen so their product fits in 16-bit integers; the released artifact's constants differ (Appendix A). Separately, Mirage v3 (§5.2) states that floating-point tests filter μGraphs with "significant numerical errors"; threshold, reference, and draw protocol are unstated.

Prism reasons over roughly 70 hand-written axioms in an e-graph. The axioms are intended to be sound; a formal proof is stated to be beyond scope. Every generated kernel is subjected to random equivalence testing with no stated tolerance, in an evaluation that is entirely half-precision. The paper does not otherwise discuss floating-point behavior [2].

Axon proves equivalence over the reals with Z3 (1650× faster than Z3's floating-point theory on its own example) and validates compiled NKI kernels on Trainium against a reference implementation on random FP32 inputs at $\texttt{rtol} = \texttt{atol} = 10^{-4}$ [3]. Axon does not state how many independent inputs the numerical gate uses; its evaluation separately reports 100 timing repetitions on random inputs, so the correctness draw count may be one, one hundred, or something else. The three mechanisms occupy different roles: a candidate-selection gate (Axon), a post-generation validation check (Prism), and a numerical-stability filter (Mirage). We measure the rule shape they share, not any one system's pipeline.

The rule has two structural properties that drive the measurements. First, it is *reference-relative*: the target is another floating-point implementation, not a higher-precision oracle, so the rule cannot observe which side of a disagreement is more accurate. Second, it is *mixed absolute-relative*. `rtol` scales with each output element, but near zero the fixed `atol` is the only protection, and accumulated rounding error grows with reduction length, not with the element's own magnitude.

3 Method

Valid corpus (registered). Six (reference, candidate) pairs, each an algebraic identity over the reals: split reduction, reassociation, scalar multiplication moved past matmul, LayerNorm two-pass vs. fused one-pass variance, naive vs. online softmax, matmul K-tiling. Identities hold symbolically. A float64 check validates the implementations: 18 cells within 6×10^{-16} to 3.5×10^{-15}. The float64 oracle is itself checked against 50-digit `mpmath` at small shapes (worst drift 8.2×10^{-16}) and assumed adequate at large ones. A negative control confirms the instrument reads nonzero.

Mutant corpus (amended). Six bug classes, eighteen instances. Four classes parameterize into the sixteen frontier instances: dropped reduction elements (j in $\{1, 2, 4, 8, 16, 32\}$), dropped contraction columns (j in $\{1, 2, 8, 32\}$), Bessel-style divisors ($n-j$, j in $\{1, 2, 8\}$), and eps added to the standard deviation instead of the variance (eps in 1×10^{-5} to 1×10^{-3}). Two gross single-instance classes run in the detection arm only. The implemented column grid is sparser than registered; the deviation is recorded in the errata. Float64 divergences span 4.1×10^{-6} to 8.2×10^{-1}. The mildest instance produces a smaller absolute disagreement than the valid rewrites themselves do (4.1×10^{-5} against 5.0×10^{-4} at $\texttt{rtol} = 0$, Appendix B), so the discrimination question is not degenerate: a threshold loose enough to admit the valid corpus is already loose enough to admit that bug.

Instrument. Oracle: the reference computed in float64 on the same rounded inputs the test-precision run sees, excluding input quantization. Per cell we record the reference's oracle error (floor), the candidate's oracle error (total), their disagreement (differential, which is what the rule tests), the direction ratio total/floor with absolute errors alongside (ratios are unstable as floor approaches zero), element exceedance fractions, and the rule verdict.

Separability analysis (E5, E8). F2's exact statements run on one fixed recorded dataset: the six valid pairs at their mlp shapes plus the K=2048 tiling cell (seven program cells), and the sixteen frontier instances. Each program contributes fifty unit-scale draws on shared seeds. For each recorded draw, the minimal accepting `atol` at each `rtol` is an explicit piecewise-linear envelope of the recorded per-element differences. Class-level envelopes under each cross-

draw semantics then decide exactly whether any (`atol` ≥ 0, `rtol`) accepts every valid program while rejecting every mutant. The construction, the four class-level envelopes, and the certificates that extend the sampled `rtol` grid to the whole domain `rtol` ≥ 0 are in Appendix B.

Protocol labels. Analyses are marked *registered* (in the original claim), *amended* (added by dated amendment before execution), or *exploratory* (single-draw sweeps that motivated later registered runs). The amendment list and the predictions that precede every run are in the committed log.

4 Findings

F1. A validator that draws its own inputs rejects a valid tiled matmul at the contraction widths of deployed models. In float32 PyTorch on the Apple M3 (CPU), under the literal elementwise rule at M=N=64 with 100 draws per cell (Table 1, Figure 1): K=4096 and 11008 are the projection and MLP contraction widths of Llama-2-7B [25]. The σ column draws both operands at the measured activation scale (σ = 2.67); the onset falls roughly fourfold from its unit-scale position, to near K=1024. Rejection is non-decreasing along both axes.

One scoping point governs how far this transfers. We draw both operands i.i.d. from the same distribution, which is what a standalone kernel validator does: Axon generates random FP32 inputs for every kernel argument, having no way to know that one of them would hold trained weights. A deployed projection or MLP matmul instead pairs an activation with a weight matrix whose entries are far smaller. The disagreement this rule measures grows with the product of the two operand scales, so that pairing pushes the onset to much larger K than the widths in the table; the σ sweep below, where scale alone moves rejection from 0 to 74 per 100 at fixed K, is the same effect on one axis. The finding is therefore about the inputs a validator generates, not a claim that a deployed Llama-2-7B matmul would be rejected. We did not measure asymmetric operand scales, and that is the measurement this finding most needs.

The rejected side is the more accurate one. At fp32 the tiled variant is 2.0× to 6.0× closer to the float64 oracle than the full-K reference it is compared against (total/floor 0.50, 0.17, 0.26 at the small, mlp, and attention shapes), so the differential the rule tests is dominated by the reference's own error. The reference here is a single library GEMM, whose internal accumulation order we do not control or characterize; F3 shows that reference choice alone can flip a verdict.

The mechanism is backend-agnostic in formulation: near-zero output elements lose the relative allowance, accumulated rounding disagreement grows with reduction length, and the any-element aggregation turns rare element violations into tensor rejection. In one diagnosed failing draw (exploratory) the violation was a single element in 4096, its magnitude 750× below the tensor median. Tensor-scale relative disagreement (maximum absolute difference over maximum oracle magnitude) stays near 10^{-6} across these cells, so the tensor-scale statistic we report does not track the verdict. Every failing draw at K=512 carried exactly one violating element, so the independence approximation over the rule's any-element quantifier reduces to the observed count and corroborates nothing by itself; what it does show is the absence of clustering, since clustered violations would have produced multi-element failures and a lower tensor rate.

K	unit scale		σ = 2.67	
512	0/100	[0, 4]	11/100	[6, 19]
1024	0/100	[0, 4]	69/100	[59, 77]
2048	0/100	[0, 4]	100/100	[96, 100]
4096	48/100	[38, 58]	100/100	[96, 100]
11008	100/100	[96, 100]	100/100	[96, 100]

Table 1: The valid K-tiled matmul under the literal elementwise rule, M=N=64, fp32, 100 draws per cell, both operands drawn i.i.d. at the stated scale. Wilson 95% CIs in percent.

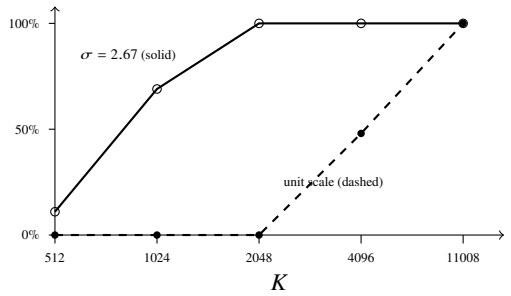

Figure 1: Rejection rate of the valid K-tiled matmul against contraction width, 100 draws per point, under unit-scale and scale-matched gaussian inputs. The onset shifts roughly fourfold with input scale.

Two same-kernel observations tie the onset to this mechanism rather than to a kernel change. At fixed K=512, input scale alone drives rejection from 0 to 74 per 100 (σ from 0.5 to 4.0: 0, 0, 1, 16, 74), where no K-correlated kernel change can act. And the recorded maximum absolute disagreement grows smoothly with K across all five widths (median 6.1×10^{-5} at K=512 to 1.3×10^{-3} at K=11008, no isolated jump), with its argmax at large, `rtol`-protected elements (D1): the binding violations are the near-zero elements.

F2. No fixed tolerance separates the recorded corpus under either extreme cross-draw semantics. However

the constants are chosen, a validator applying either extreme cross-draw rule cannot accept every valid rewrite while catching all sixteen injected bugs on the recorded data. The recorded errors are separable, but only by an assignment that already knows which programs are the bugs.

Exact result. On the separability corpus, with the envelopes of Appendix B: under all-draws-must-pass (a candidate is accepted iff every draw passes), no ($\texttt{atol} \geq 0$, $\texttt{rtol} \geq 0$) separates, with supremum separation gap -4.0×10^{-6}. Under any-draw-may-pass (accepted if any draw passes), the same, at -4.2×10^{-6}. The pessimal pairing, valid programs held to every draw while mutants must fail every draw, reads -3.5×10^{-5}. All 400 inter-sample intervals in each analysis carry the single-witness certificate, and the mutant-side envelopes are nonpositive at $\texttt{rtol} = 100$ (-5.5×10^{-5} and -2.4×10^{2}), which extends all three results to the whole domain $\texttt{rtol} \geq 0$.

The pairing that separates. The fourth pairing is a class-conditioned information bound, not an operational validator. It judges each valid program on its most favorable recorded draw and each mutant on its least favorable one, an assignment that requires knowing the class in advance. Under it, 110 consecutive sampled rtol values separate, spanning $[7.8 \times 10^{-3}, 7.9]$, with maximum margin $+9.5 \times 10^{-6}$ (for example, $\texttt{atol} = 10^{-6}$ at $\texttt{rtol} = 0.056$). Our interval certificate is one-sided: it certifies non-separation, so the interior of that band is sampled rather than certified. This falsified our registered prediction that all four pairings would fail to separate, and it makes F4 concrete: which draws a program is judged on can decide separability outright. The choice of per-class semantics, which no source paper states, swings the margin from -3.5×10^{-5} across zero to $+9.5 \times 10^{-6}$. Intermediate Boolean rules (k-of-n acceptance) evaluate the k-th order statistic of the same per-draw envelopes. They are legitimate rules and another unstated coordinate; we do not analyze them here.

On the 64-point grid. Total error is the equal-weight sum of the valid-rejection and mutant-miss fractions; the weighting is an analytical choice, and Figure 2 shows the two components separately. Its minimum on the grid is a three-point plateau: $(10^{-4}, 10^{-4})$, $(10^{-4}, 10^{-3})$, and $(10^{-4}, 10^{-2})$. Each plateau point reads 0% valid rejection (unit scale) plus a 6.25% mutant-instance miss fraction, which is 1 of the 16 frontier instances, equivalently 50 of 800 mutant draws. The missed instance is the eps-10^{-5} bug, which is also the binding instance in the exact analysis. This fraction describes the constructed corpus, not any candidate population. Axon's published constant lies on that plateau, and it clears the valid corpus narrowly: at the grid rtol nearest 10^{-4} the valid side requires $\texttt{atol} = 9.4 \times 10^{-5}$ against the published 10^{-4}, so the 0% rejection figure is one draw from moving. The binding coordinate is atol.

The evading bug. The eps bug's severity is input-dependent (it grows as row variance approaches eps), so detection statements hold at the measured input statistics.

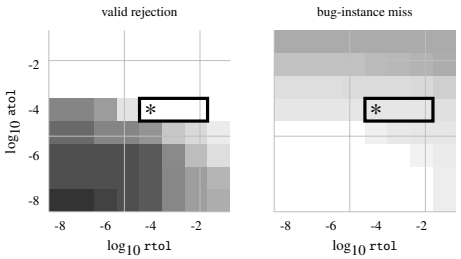

Figure 2: Component error fractions over the 64-point (atol, rtol) grid, 50 draws per program: left, valid-rewrite rejection; right, bug-instance miss. Darker is worse on one absolute scale. Outlined: the minimum-error plateau of the equal-weight sum on the tested grid (0% valid rejection, 6.25% miss). Star: Axon's published constant. The exact envelope computation (E5, E8) extends the no-separator result to the continuum over $\texttt{rtol} \geq 0$ under the two extreme Boolean cross-draw semantics analyzed.

Under scale-matched gaussian inputs at σ 2.67 the picture is unchanged: the five other detection-arm mutants at 100/100 detection, the eps bug evading all 100 draws, its divergence shrinking to 2.46×10^{-6} as the registered mechanism predicts. An oracle-referenced check with a condition-aware budget could see it where the sampled rule cannot. We do not demonstrate that a principled budget resolves 4.1×10^{-6} at these statistics.

F3. Reference choice changes the verdict and the direction of the accuracy comparison. Observed at fp16 on the registered cell at (512, 1024), for the same online-softmax candidate (oracle error 6.0×10^{-4} on the registered draw), Table 2. "Closer" and "farther" give the candidate's accuracy against the float64 oracle relative to the reference it is compared with. The verdict flips with the reference, and the direction against the tree reference is itself draw-dependent: closer on the registered draw, farther in the 100-draw mean. The mean floors order seq > tree > torch.sum, but the per-draw ordering holds on only 80/100 (fp16) and 85/100 (bf16) draws, so the ranking of references is also draw-dependent. At bf16 the single-draw ratios are 1.92, 2.14, and 0.02, and the rule fails all three on 100/100.

torch.sum's smaller floor is consistent with a different internal accumulation strategy, which we do not characterize here; FPRev shows that such accumulation orders are implementation properties and are often undocumented [7]. A reference-relative rule observes disagreement but not its direction with respect to oracle accuracy. The cells where our two gate definitions disagree are exactly the cells where the candidate is closer to the oracle than its reference.

F4. Detection is a per-draw rate, and the source papers do not fully specify the draw count or how draws are

reference	floor	registered draw	100-draw mean	verdict, 100 draws
sequential	8.9×10^{-3}	15× closer	11× closer	fails 100/100 [96–100]
strided tree	9.7×10^{-4}	1.6× closer	1.7× farther	passes 98/100 [93–99]
`torch.sum`	3.7×10^{-4}	1.6× farther	2.3× farther	passes 99/100 [95–100]

Table 2: The same online-softmax candidate at fp16, (512, 1024), against three references. Floor: the reference's own oracle error on the registered draw (candidate: 6.0×10^{-4}). "Registered draw" and "100-draw mean": the candidate's oracle accuracy relative to that reference. Wilson 95% CIs in percent.

aggregated. Observed: the valid K=512 tiling trips the rule on 14/100 draws [9–22%] under gaussian inputs at the measured activation scale ($\sigma = 2.67$) and 0/100 at unit scale (upper bound 4%). Two further registered runs of the same cell, differing only in seed stream and in whether the draws are clamped to the observed activation range, read 11/100 [6–19%] (the K table) and 16/100 [10–24%]. The spread across those three runs is itself the point: a single 100-draw estimate of this rate carries a several-point interval. Our own exploratory single-draw sweep missed the failure; the registered repeated-draw run caught it.

Two aggregation questions govern what such rates mean. Within a tensor, Axon's rule is elementwise and unambiguous: one violating element rejects the tensor. Across random inputs, everything is unstated: the number of draws, whether a candidate must pass all of them, and how errors aggregate. The next two figures are hypothetical consequences under IID draws and an all-draws-must-pass rule, not measurements of any system. A valid rewrite with a 14% per-draw rejection rate survives 20 draws with probability near $0.86^{20} \approx 4.9\%$. A 0/100 observation is consistent both with a true catch probability of zero (more draws never help) and with a rate at its 3.7% upper bound, where 100 draws would catch the bug with probability near 97.7%. A reported draw count without its cross-draw aggregation rule is still not enough to reproduce a validator.

F5. An FP32-class fixed tolerance does not transfer to reduced precision. At the mlp shape, an untransformed matmul deviates from exact arithmetic on its own inputs by 3.91×10^{-4} at fp16 and 3.45×10^{-3} at bf16, measured scale-relative (maximum absolute difference over maximum oracle magnitude). All 36 reduced-precision cells fail a 10^{-4}-class rule.

The direction ratio shows that reference and precision error dominate these failures, not the rewrites. Against their sequential references, the three accumulation-reordering pairs (split reduction, reassociation, online softmax) are 3× to 73× closer to the oracle than the reference they fail against, and 8× to 62× closer at the mlp shapes. Axon scopes its constant to FP32 explicitly, and Prism's benchmarks are half-precision with no stated threshold. This characterizes what would happen if an FP32-class threshold were transferred, not what any system's actual criterion does.

F6. Real activations are benign in this model. Observed: every real-activation FP32 cell passes, including post-LayerNorm row sums at condition number 368,927, 400× beyond unit-gaussian reach (condition rose 400×, error rose 11×). We do not quantify the margin. The tensor-scale statistic suggests a wide one, but F1 shows that statistic does not track this rule: in two of these cells individual elements reach relative disagreements of 1.4×10^{-1} and 2.4×10^{-1} and pass only because `atol` carries them, so the elementwise margin at the binding element may be near 1. Our records do not store a per-cell gate margin, so the claim here is the verdict, not a distance from it. The fused-variance hazard does not materialize because the captured residual stream is near zero-mean in the bulk of its rows. Per-row cancellation, measured as |mean| over mean$|x|$, reaches at most 0.012 at the 99th percentile and 0.16 in the worst row, across the two residual captures. This is an observation about this 6-layer pre-norm model, not an architectural guarantee.

Matching scale is not matching distribution: the scale-matched gaussian draws that reject in F1 share only variance with these activations, and the real tensors pass. The evidence base is one small character-level model and one batch; broader pretrained-model coverage is the principal remaining external-validity limitation.

F7. Inputs that force valid-corpus failures are far outside the observed activation distribution. Constructed cancellation inputs trip the rule on every draw, but their median row condition number is 4.5×10^6 against 42.8 for real rows (10.4σ on a log scale), and the elementwise violations are near 2× tolerance.

Online softmax is immune under every strategy measured. It is also, by our inference from the verifiers' mechanisms, the pair they handle worst: its running max is not a Lax operator, so Mirage partitions around it, and Axon's uninterpreted exp blocks the rescale identity. Neither paper classifies the transformation itself. Exceptional regimes (NaN, Inf, subnormals, overflow) are not covered by our gaussian and activation inputs and remain open.

5 Threats to validity

Backend and reimplementation (F1–F5). No emitted kernel from any system is executed; PyTorch reimplementations stand in. Headline K and scale grids run in float32 on the Apple M3 CPU; corpus equivalence and hardware characterization are verified on an Ampere A10. Measured onsets (F1) and margins (F2) are backend-conditioned, and FPRev and FTTN document that real accumulation orders and extra-precision behavior vary across libraries and accelerators.

Mutant severity is input-dependent (F2). Detection rates are statements at the stated input statistics.

Reference coverage (F3). Three references demonstrate the effect on one backend; production references may use blocked or multiway accumulation orders that differ from the three measured here. The verdict flip is carried by the sequential reference: the two library-plausible references agree with each other, passing the candidate on 98/100 and 99/100 draws, and both read "farther" in the 100-draw mean. Admitting a 1024-term sequential fp16 sum as a reference is what produces the flip, and whether a real system would use one is not established. F3 is also measured only at fp16 and bf16, where F5 shows a 10^{-4}-class constant does not apply; at fp32 the three references very likely agree.

Corpus composition (F2). The envelopes are extreme order statistics over unequal populations: per draw the valid programs contribute about 3.7M output elements and the mutants about 0.4M, and the classes are not evaluated at each other's shapes. Both monotonicities run the same way, since adding mutant instances only lowers the mutant-side envelope and adding valid draws or larger tensors only raises the valid-side one, so a wider corpus can only make non-separation easier to obtain. The binding mutant throughout is the mildest instance of a family we designed as a severity continuum; a milder floor would push the gap further negative and a coarser one might separate. The non-separation is also cross-operator, pooling a 3072-wide matmul tiling with a 1024-wide LayerNorm bug, and we do not report per-operator separability, which is the form of the fix Section 6 recommends.

One model, one batch (F6). A pretrained-transformer activation corpus across layers and batches is the single most valuable missing dataset.

Oracle and ratios. The float64 oracle is validated at small shapes, and the F3 headline cell is spot-checked against a 50-digit oracle on three draws, worst relative deviation 1.7×10^{-13} (D2); direction ratios are reported with absolute errors. Exploratory single-draw cells exist and are labeled; headline claims rest on registered repeated-draw runs.

Selection amplification (not measured). A validator inside a superoptimizer is queried repeatedly by an adaptive candidate-generation process, so rare misses may be amplified by selection. Quantifying that requires a candidate distribution or an actual search loop; it is the natural next registered experiment, and the mutant-instance miss fraction above is not an estimate of any candidate-population miss probability.

6 Implications

For Axon. On our corpus, Axon's constant sits on the minimum-error plateau of the tested 64-point grid (F2). Its proofs are shape-generic, but its rule's verdict becomes shape-dependent within deployed contraction widths on our backend (F1). The measurements support reporting three things: the draw count, the reference implementation and its accumulation order, and a justified error budget for the operator, shape, and precision, of the kind derived for mixed-precision GEMM by Blanchard et al. [16] and Higham and Mary [17]. A reference-relative rule cannot tell whether a disagreement improves or worsens oracle-relative accuracy; an oracle comparison could preserve accuracy-improving candidates (F3).

For Prism. Half-precision evaluation with an unstated threshold sits where F5 shows fixed FP32-class constants do not transfer; a stated and justified tolerance for its half-precision regime would make its random testing interpretable.

For Mirage. Mirage v3 already places a numerical-stability filter after its algebraic verification. Stating the filter's threshold, reference, and draw protocol would let a reader reproduce it from the paper alone. Appendix A records that the filter is not locatable in the released artifact at the audited commits.

Method integrity. Our own results required correction repeatedly during this study, and every correction moved against our own claims; the errata records each with its cause. The most instructive was a K=2048 rejection verdict produced by comparing a maximum absolute difference against `atol` alone, the same quantity confusion this paper studies.

7 Related work

TASO [5] began validation-adjacent practice in tensor superoptimization, using random-tensor execution to identify

candidate-equivalent graphs before formally verifying substitutions. TensorRight [4] develops stronger idealized rewrite verification; Mirage, Prism, and Axon combine stronger semantic reasoning with residual empirical validation of generated implementations. TTrace [6] is the closest adjacent study. In distributed-training validation it shows that fixed `torch.allclose` tolerances produce false positives, false negatives, or both, and it replaces them with perturbation-derived dynamic tolerances. Our subject differs: superoptimizer acceptance rules, real-equivalent rewrites and injected bugs, and reference and shape sensitivity. A concurrent line of 2026 measurements by one author (Sarkar) reaches adjacent conclusions on neighboring workloads: fixed-shape small-sample allclose oracles pass seeded-bug GPU kernels [22], per-operator and per-dtype tolerance calibration trades detection against false positives [23], and input-generation strategy materially changes which kernel bugs surface [24]; none studies the superoptimizer acceptance stage or reference sensitivity.

FPRev [7] infers the undocumented accumulation orders of real libraries and accelerators, and FTTN [8] shows accelerator numerical features are under-documented across vendors; both indicate that the reference sensitivity in F3 is present in deployed libraries and accelerators. LifeJacket [12] and Alive-FP [11] verify LLVM floating-point optimizations under actual FP semantics and found real incorrect optimizations; Minotaur [13] brings formal FP reasoning to SIMD superoptimization. NNSmith [14] treats input generation as central to exposing compiler bugs, as our scale and distribution findings do for validation.

Herbie [10] rewrites for accuracy, which is exactly the directional information reference-relative rules lack. FPBench [15] standardizes accuracy measures for FP tools, the measurement-definition problem that our oracle and metric choices also face. Rounding-error analyses (Goldberg [19]; Higham [20]; SATIRE [18]; Blanchard et al. [16]; Higham and Mary [17]) derive the accumulation-dependent budgets a specified rule could draw on. Mytkowicz et al. [21] showed that performance evaluation produces wrong conclusions from unexamined setup; this paper applies the same scrutiny to correctness validation.

8 Conclusion

A numerical validation result is interpretable only relative to a specified validation protocol. Individual protocol coordinates materially change the verdict: contraction length and output multiplicity (F1), tolerance constants and the cross-draw semantics (F2), reference implementation (F3), draw count (F4), precision (F5), and input scale and distribution (F1, F6, F7). On the recorded corpus, no nonnegative mixed absolute-relative tolerance separates real-equivalent rewrites from injected bugs under the two extreme cross-draw semantics analyzed, at any `rtol` ≥ 0. Among the semantics we analyzed, only a class-conditioned draw assignment separates it, and that assignment requires knowing the class in advance; intermediate k-of-n rules are uniform and remain unanalyzed. No evidence indicates that any system has shipped an incorrect kernel.

We propose a reporting standard. A floating-point acceptance result should specify at least: (1) the reference implementation and its relevant accumulation behavior; (2) the precision; (3) the input distribution and scale; (4) the per-element comparison function and constants; (5) the within-tensor aggregation rule; (6) the number of independent draws; (7) the cross-draw aggregation rule; and (8) a justification for the numerical error budget in the operator, shape, and precision regime.

This paper does not prescribe one universal validator. It identifies the information needed to make a validator's result reproducible and interpretable, and measures what remains undetermined when that information is absent.

Artifact. All measurements reproduce from the public repository; each experiment runs from one script, raw per-cell records and the activation fixtures are committed, and the pre-registration, amendments, run log, and errata are public. One script, `tools/check_paper_numbers.py`, checks the paper's numbers against those records; it resolves a handful of them against quoted lines of the committed run log rather than against a data file, and labels which. AI tools (Claude) assisted with code, measurements, and documentation; the developer directed, reviewed, and verified all work, and no measured number comes from a model.

Appendix A: implementation observations on the Mirage artifact

Exploratory; the checklist of Section 8 applied to a released artifact. Audited: `mirage-project/mirage` at commit `5c28cc6`, the tip of the default `mpk` branch (2026-07-28), with the `evaluation` branch and the older `main` branch (`ffe38df`, 2026-04-17) cross-checked; method, search terms, and positive controls in `audits/mirage-fp-filter.md`. The released `superoptimize` path is search, exact finite-field fingerprint verification, transpilation, then performance-only selection over random inputs; we found no floating-point comparison in the audited acceptance path at these commits, which leaves the tolerance, precision, and input-distribution coordinates unexercised there, while the per-element rule (exact field equality), reference (input-graph fingerprints), draw count (a single fingerprint evaluation per candidate, consistent with the paper's acknowledged single test), and aggregation (all outputs must match) are determined by code. The numerical-stability filter described in the paper's v3 (§5.2) is not locatable in the released search path at these commits.

The closest thing to it is a CI test, `is_closed()` in `tests/python/test_tensor_program.py`, which compares hand-constructed graphs against torch references and counts an element mismatched only when relative *and* absolute error both exceed 10^{-1}. We record it because a reader will find it: it is not the filter, since it never calls `superoptimize()`, never reaches the verifier, and rejects no candidate, and its conjunctive criterion is looser than an `allclose` rule. Other tolerances (demo scripts, transpiler tests) sit equally far from the acceptance path. The audit also covered the published camera-ready (identical sentence, no appendix), the artifact-evaluation instructions that earned the Results Reproduced badge (performance and search time only, no numerical check), the group's later papers, the repository documentation, and the issue and discussion history; the protocol is stated in none of them. Two recorded talks were not transcribed, so a threshold named aloud would not have been seen.

The finite-field constants are `FP_P=167`, `FP_Q=83` on all three audited branches, while the paper states p=227, q=113. Theorem 2 of the paper bounds a single test's acceptance probability by $8dk^4/q + q^{-1/k^2}$; both terms grow as q falls, so the released q loosens the stated bound at every fixed (d, k) by a factor that depends on (d, k). An earlier 1.4× reading treated the bound as proportional to $1/q$ and is retracted in the errata. These are observations about the released repository at pinned commits, not about any deployed system.

Appendix B: envelope construction and certificates

The separability corpus (Section 3): seven valid program cells and sixteen mutant instances, fifty unit-scale draws per program on shared seeds. The two gross detection-arm instances are excluded; adding mutant instances can only lower the mutant-side envelope, so non-separation extends a fortiori to supersets.

For a recorded draw with per-element differences $d_i = |g_i - b_i|$ and reference magnitudes $x_i = |b_i|$, let $T(r) = \max_i(d_i - r\,x_i)$. This is the unclamped absolute slack the draw requires at relative tolerance r: any real $\texttt{atol} \geq T(r)$ accepts. The rule restricts $\texttt{atol} \geq 0$, so the smallest admissible accepting threshold is $\max(T(r), 0)$, the clamp the separator test applies. Each T is a finite maximum of lines through recorded points, evaluated exactly on the Pareto set of the (x_i, d_i); it is convex, piecewise linear, and non-increasing.

Four class-level envelopes cover the extreme Boolean per-class semantics:

- $F_{\text{all}}(r)$: the maximum over all valid draws. A valid program is accepted only if every draw passes.
- $F_{\text{any}}(r)$: the maximum over valid programs of the minimum over each program's draws. A valid program is accepted if some draw passes.
- $G_{\text{every}}(r)$: the minimum over all mutant draws. A mutant is caught only if every draw fails.
- $G_{\text{some}}(r)$: the minimum over mutant instances of the maximum over each instance's draws. A mutant is caught if some draw fails.

A separator exists at r for a pairing (F, G) iff $\max(F(r), 0) < G(r)$.

The computation samples 401 `rtol` values (zero and 400 log-spaced in $[10^{-9}, 100]$) and certifies each inter-sample interval with a single-witness bound. The witness is a draw envelope, or for G_{some} a witness instance's max-envelope (itself a maximum of convex functions). Either is convex and so lies below the larger of its endpoint values on the interval, while every F is non-increasing and so lies above its right-endpoint value. When the witness bound does not exceed $\max(F, 0)$ at the right endpoint, no r in the interval separates. Convexity is used only for single witness envelopes, never for the min-envelopes G, which need not be convex. Every envelope is non-increasing, so a nonpositive mutant-side envelope at $r = 100$ extends no-separation to all $r > 100$, closing the domain $\texttt{rtol} \geq 0$.

The certificate is one-sided: it establishes non-separation on an interval, never separation. Where separators appear, we report the sampled values that separate and do not claim the interior between them.

References

[1] M. Wu et al. Mirage: A Multi-Level Superoptimizer for Tensor Programs. OSDI 2025. arXiv:2405.05751 (v3).

[2] M. Wu, X. Jiang, O. Padon, Z. Jia. Prism: Symbolic Superoptimization of Tensor Programs. arXiv:2604.15272.

[3] A. Kothari, S. Zhu, D. Kroening, C. Sung. Axon: A Synthesizing Superoptimizer for Tensor Programs. arXiv:2606.26344.

[4] J. Arora et al. TensorRight: Automated Verification of Tensor Graph Rewrites. PACMPL 9 (POPL), 2025.

[5] Z. Jia, O. Padon, J. Thomas, T. Warszawski, M. Zaharia, A. Aiken. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. SOSP 2019.

[6] Jiang et al. TTrace: Lightweight Error Checking and Diagnosis for Distributed Training. arXiv:2506.09280.

[7] Xie et al. Revealing Floating-Point Accumulation Orders in Software/Hardware Implementations. USENIX ATC 2025. arXiv:2411.00442.

[8] X. Li et al. FTTN: Feature-Targeted Testing for Numerical Properties of NVIDIA and AMD Matrix Accelerators. arXiv:2403.00232.

[9] C. Nandi, M. Willsey, et al. Rewrite Rule Inference Using Equality Saturation. OOPSLA 2021.

[10] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, Z. Tatlock. Automatically Improving Accuracy for Floating Point Expressions. PLDI 2015.

[11] D. Menendez, S. Nagarakatte, A. Gupta. Alive-FP: Automated Verification of Floating Point Based Peephole Optimizations in LLVM. SAS 2016.

[12] A. Nötzli, F. Brown. LifeJacket: Verifying Precise Floating-Point Optimizations in LLVM. SOAP 2016.

[13] Z. Liu, S. Mada, J. Regehr. Minotaur: A SIMD-Oriented Synthesizing Superoptimizer. OOPSLA 2024.

[14] J. Liu et al. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. ASPLOS 2023.

[15] N. Damouche, M. Martel, P. Panchekha, C. Qiu, A. Sanchez-Stern, Z. Tatlock. Toward a Standard Benchmark Format and Suite for Floating-Point Analysis. NSV 2016.

[16] P. Blanchard, N. J. Higham, F. Lopez, T. Mary, S. Pranesh. Mixed Precision Block Fused Multiply-Add: Error Analysis and Application to GPU Tensor Cores. SIAM J. Sci. Comput. 2020.

[17] N. J. Higham, T. Mary. A New Approach to Probabilistic Rounding Error Analysis. SIAM J. Sci. Comput. 2019.

[18] A. Das et al. Scalable yet Rigorous Floating-Point Error Analysis. SC 2020.

[19] D. Goldberg. What Every Computer Scientist Should Know About Floating-Point Arithmetic. ACM Computing Surveys, 1991.

[20] N. J. Higham. Accuracy and Stability of Numerical Algorithms. SIAM, 2002.

[21] T. Mytkowicz, A. Diwan, M. Hauswirth, P. F. Sweeney. Producing Wrong Data Without Doing Anything Obviously Wrong! ASPLOS 2009.

[22] D. Sarkar. The Correctness Illusion in LLM-Generated GPU Kernels. arXiv:2606.20128.

[23] D. Sarkar. Operator-Aware Mixed-Precision Tolerance Calibration for Tensor Kernels. arXiv:2607.16228.

[24] D. Sarkar. Test-Input Generation for Tensor Programs: What Actually Finds Kernel Bugs. arXiv:2606.27396.

[25] H. Touvron et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023.